Module 17: AAP Best Practices

Duration	Theory
3 Hours	50%
Hands-on	
50%	

After completing this module, participants will be able to design production-ready AAP architectures, follow secure automation practices, organize content effectively, implement scalable infrastructure, and avoid common automation pitfalls.

17.1 Architecture and Planning Best Practices

Capacity Planning Guidelines

Capacity planning ensures AAP infrastructure can handle automation workloads. Key factors: number of managed hosts, concurrent jobs, playbook size, and job frequency.

Example Scenario

- 500 servers managed
- 5-minute average job duration
- 10 concurrent jobs required

Controller

4–8 CPU cores

Memory

16–32 GB RAM

Database

Dedicated PostgreSQL

Execution

Multiple nodes

Planning Example

Small Environment

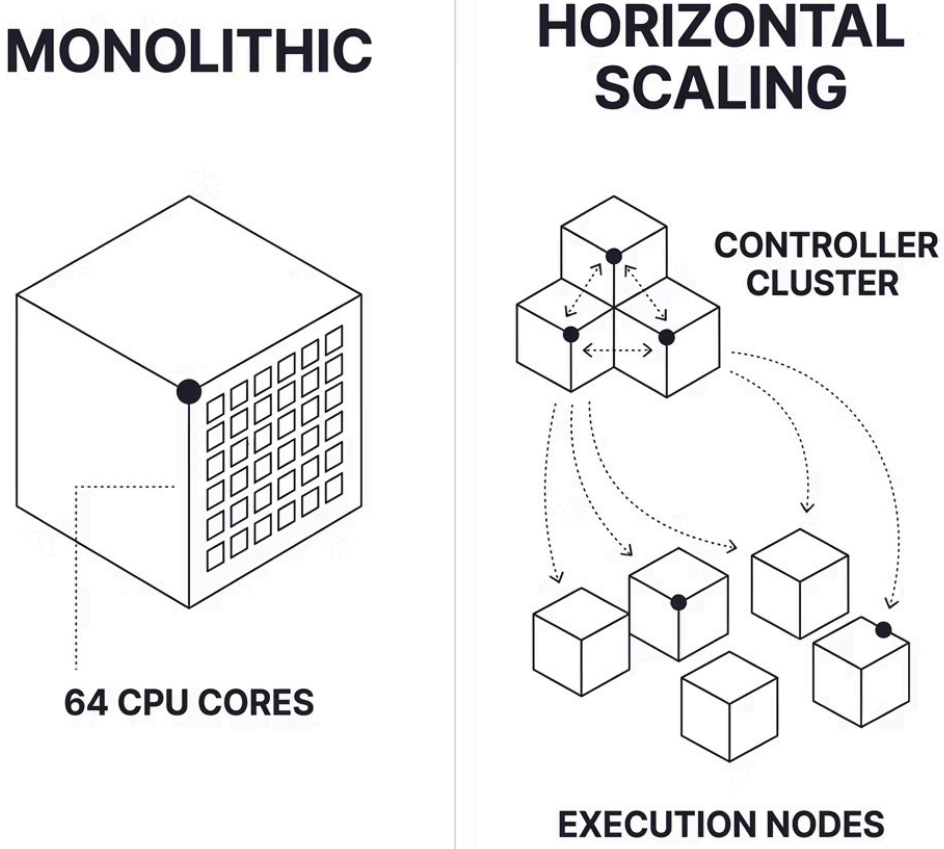
- 1 Controller
- 1 Execution Node

Large Environment

- 2 Controllers (HA)
- 4 Execution Nodes
- 1 Dedicated Database

Proper planning prevents **automation delays and system overload**.

When to Scale Horizontally

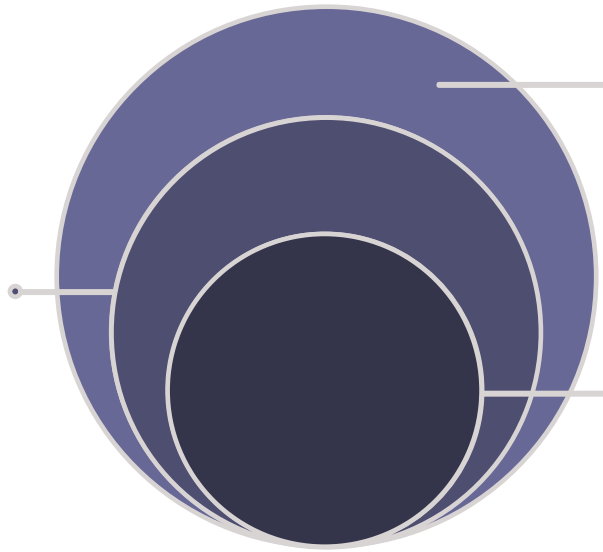


Benefits of Horizontal Scaling

- Improved performance
- Higher reliability
- Better job distribution

AAP supports **execution node clusters** — add more nodes instead of upgrading hardware.

High Availability Considerations



Why HA Matters

High availability ensures automation continues even if a component fails.

No Single Point of Failure

Uninterrupted Automation

Disaster Recovery Basics

1

Backup Strategy

Daily database backups, weekly config snapshots, container image backups

2

Restore Database

Recover PostgreSQL from latest backup

3

Redeploy Controllers

Bring controller nodes back online

4

Reconnect Nodes

Reconnect execution nodes to restored infrastructure

Network Design Recommendations

AAP requires reliable network communication across all layers of the automation stack.

Port 22

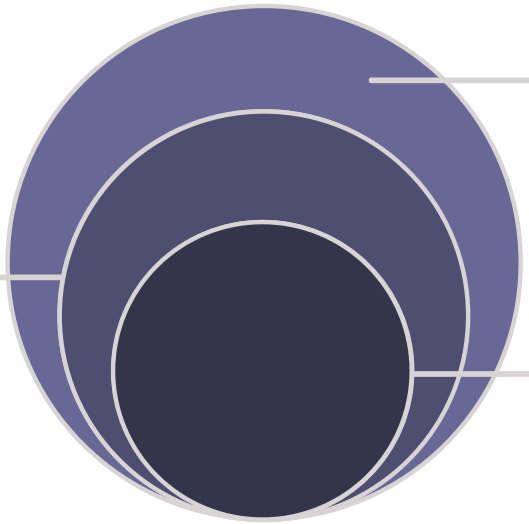
SSH connections

Port 443

Controller API

Port 5432

PostgreSQL database



Key considerations: low latency, secure firewall rules, segmented automation networks.

Database Maintenance & Backup Strategies

AAP Uses PostgreSQL

The database stores job history, credentials, inventory data, and automation configuration.

Best Practices

- Daily backups
- Database vacuum operations
- Log cleanup

Example: `pg_dump awx > backup.sql`

17.2 Development and Content Best Practices

Organizing Playbooks and Roles Effectively

Recommended Project Structure

```
automation-project
 ├── playbooks
 ├── roles
 ├── inventories
 ├── collections
 └── tests
```

Good organization improves **maintainability** and team collaboration.

Example Layout

```
playbooks/
  deploy_app.yml
roles/
  nginx/
  database/
```

Playbooks

Task orchestration

Roles

Reusable components

Tests

Validation layer

Naming Conventions for Clarity

Use Clear, Descriptive Names

Playbook

`deploy_nginx.yml`

Role

`nginx_install`

Variable

`web_server_port`

Avoid Unclear Names

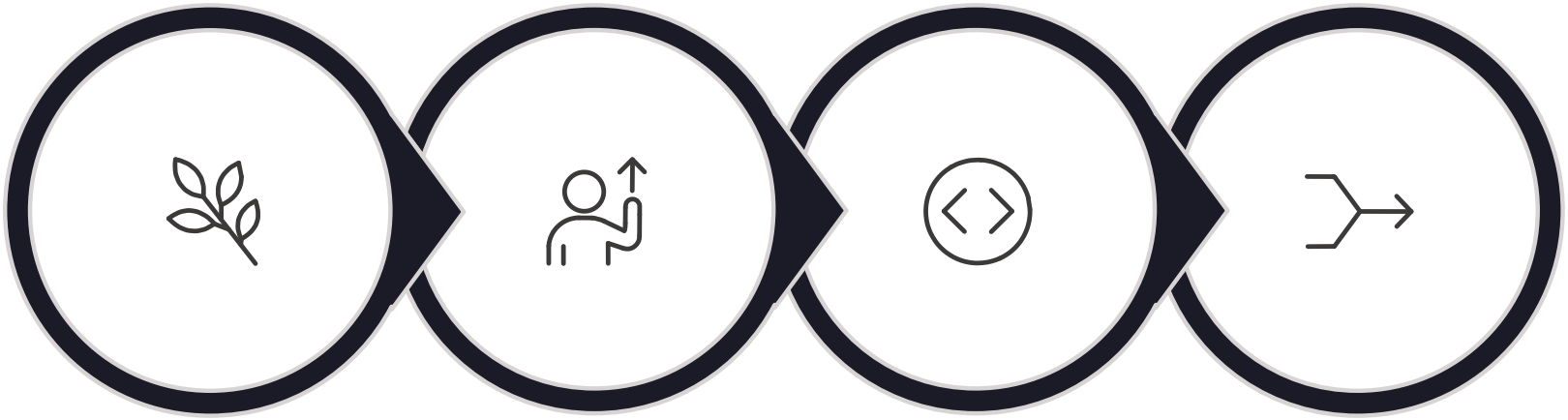
❌ Bad

`playbook1.yml`
`test_script.yml`

Clear naming helps **team collaboration** and long-term readability.

Version Control Workflow

All automation content should be stored in **Git repositories**. Automation Controller integrates natively with Git-based projects.



Benefits: change tracking, rollback capability, and team collaboration.

Testing Before Production

1

Development

Write & iterate

2

Testing

Validate logic

3

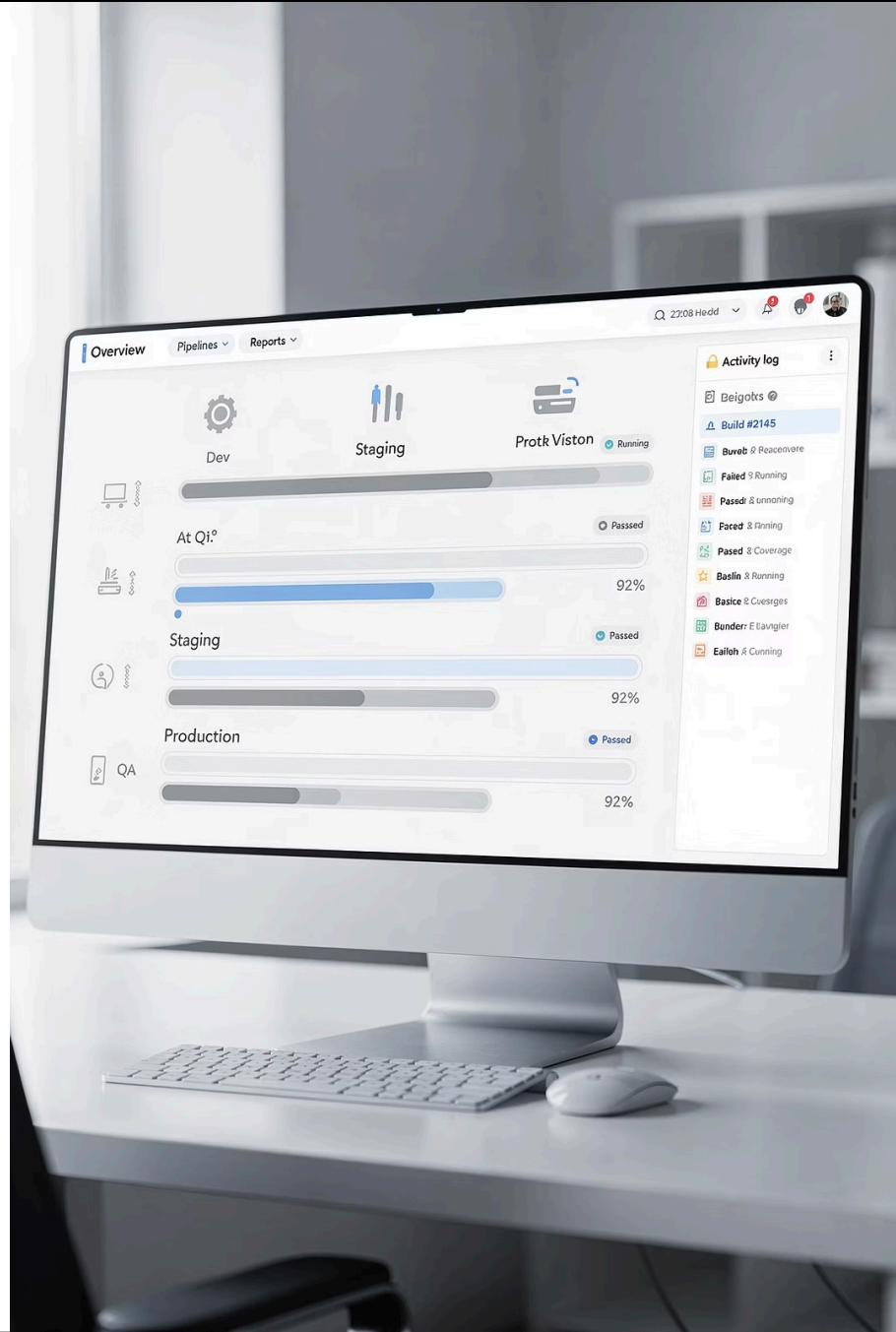
Staging

Pre-prod check

4

Production

Safe deployment



Code Review Processes

Review Checklist

- → Syntax Correctness
- → Idempotency
- → Security Issues
- → Variable Handling

> *"Will this playbook run safely multiple times?"*

Documentation Standards

What to Document

- Playbook purpose
- Required variables
- Execution instructions

Good documentation improves **team knowledge sharing**.

Example: `deploy_nginx.yml`

```
Purpose:
 Deploy nginx web server

Variables:
 nginx_port
 nginx_version
```

17.3 Security Best Practices

Credential Management Principles

Credentials should **never be stored in plain text**. AAP provides secure, encrypted credential storage in the database.

SSH Credentials

Encrypted keys for host access

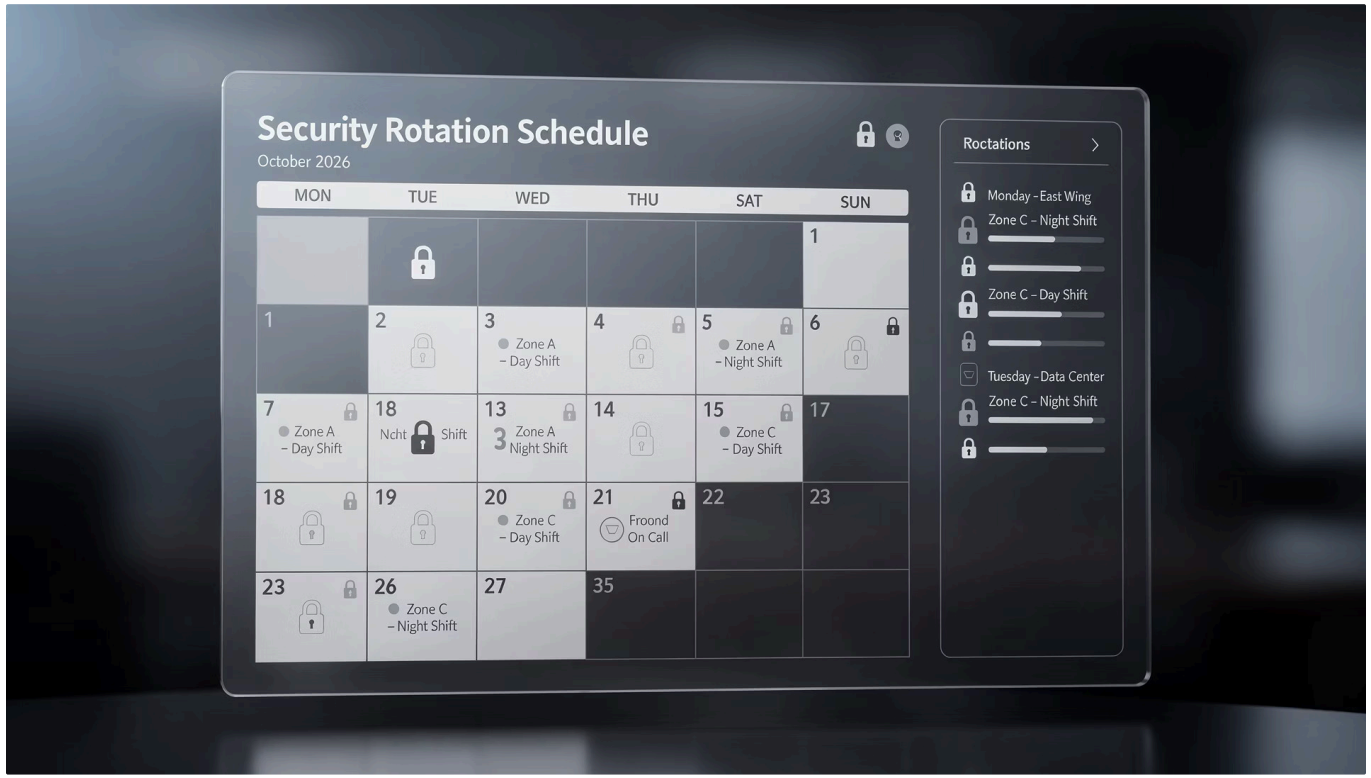
Cloud API Keys

Secure tokens for cloud providers

Database Passwords

Protected connection credentials

Regular Credential Rotation



Rotation Policy

SSH Keys

Rotated every **90 days**

API Tokens

Rotated **monthly**

Regular rotation reduces **security risks** from compromised credentials.

Implementing Least-Privilege RBAC

RBAC ensures users only have the permissions they require. Least privilege prevents **unauthorized changes**.

Admin

Full access

Operator

Run jobs

Developer

Create playbooks

Auditor

Read-only access